

Guía Docente — Sesión 1: Introducción, Virtualización y Primeros Contenedores

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026) **Docente:** Ing. Cristian Jampier Chileno Segundo | OTI - UNI **Programa:** Programa de Iniciación Tecnológica (PIT) 2026

🌟 INTRODUCCIÓN PEDAGÓGICA AL CURSO

¿Qué es Docker? (La explicación definitiva para los alumnos)

Imaginen que quieren enviar un regalo a un amigo en otro país. En lugar de enviarle las piezas sueltas y esperar que tenga las herramientas correctas, ustedes meten el regalo completamente armado dentro de una caja estándar. Docker es esa caja. En el mundo del software, llamamos a esta caja **Contenedor**. Docker permite empaquetar código fuente, librerías, variables de entorno y el sistema operativo mínimo, todo dentro de una unidad ligera e inmutable llamada **Imagen**.

¿Cómo funciona por debajo?

A diferencia de las máquinas virtuales que cargan un SO completo, Docker comparte el Kernel del sistema operativo anfitrión (Host). Utiliza **Namespaces** (para aislar procesos, red, sistema de archivos y usuarios) y **Control Groups o cgroups** (para limitar y medir los recursos: RAM, CPU e I/O).

¿Para qué usar Docker? (Los beneficios reales)

1. **Portabilidad absoluta:** Si corre en tu laptop, correrá exactamente igual en producción.
2. **Arranque en milisegundos:** Es solo un proceso aislado, no un SO completo emulado.
3. **Ahorro de infraestructura:** Decenas de contenedores en un servidor donde antes solo cabían 2-3 VMs.
4. **Facilidad de actualización:** Reemplazar el contenedor por uno nuevo en segundos.

🗺 El Camino del Curso (Ruta de Aprendizaje)

- **Sesión 1 — Fundamentos:** Comandos básicos para encender, apagar y listar contenedores.
- **Sesión 2 — Empaquetamiento:** Escribir Dockerfiles optimizados.
- **Sesión 3 — Orquestación:** Unificar contenedores (Web + BD) con Docker Compose.
- **Sesión 4 — Persistencia y Redes:** Aislar bases de datos, crear redes privadas y backups.
- **Sesión 5 — Producción:** Proxies inversos (Nginx), Multi-stage builds y limitar recursos.
- **Sesión 6 — Automatización:** Scripts de despliegue continuo y proyecto final integrador.



Perfil del Alumno y Enfoque Pedagógico

Los estudiantes son administradores de sistemas y desarrolladores junior que conocen arquitecturas tradicionales: servidores físicos, máquinas virtuales clásicas (VMware, VirtualBox), instalación directa de dependencias. No poseen conocimientos previos sobre aislamiento a nivel de kernel. Iniciar con la analogía del transporte de Malcolm McLean (1956) para conectar el problema del software con una solución tangible.

Planificación de la Clase (3 Horas)

Bloque	Tiempo	Contenido
Bienvenida	00:00 - 00:20	Presentación del docente, programa PIT y mapa del curso
Bloques 1 & 2	00:20 - 00:50	Introducción y Fundamentos, Virtualización e Hipervisores
Bloque 3	00:50 - 01:20	Instalación de Docker (Engine/Desktop)
Bloque 4	01:20 - 01:40	Imágenes y Contenedores
Receso	01:40 - 01:55	Break
Bloques 5 & 6	01:55 - 02:40	Comandos Esenciales y Primera Aplicación Flask
Cierre	02:40 - 03:00	Talleres Complementarios, Tarea y Q&A

Guión Paso a Paso del Docente

Introducción: Cómo comenzar la clase

Guión Sugerido (Lo que debes decir): *"Muy buenos días con todos. Bienvenidos a la primera sesión del curso 'Docker desde Cero: Crea y Despliega Aplicaciones' del PIT 2026 de la UNI. Mi nombre es Cristian Jampier Chileno Segundo y seré su docente. Hoy comenzaremos desde cero absoluto. Entenderemos de dónde viene la tecnología de contenedores, por qué es superior a las máquinas virtuales clásicas y daremos nuestros primeros pasos en la terminal."*

Bloque 1: Introducción y Fundamentos (El dilema del software)

Guión Sugerido (Lo que debes decir): *"Antes de 1956, cargar un barco mercante era un caos: sacos de azúcar, cajas y barriles de todos los tamaños se estibaban de forma manual. Malcolm McLean revolucionó el transporte con el contenedor metálico estándar: cajas de dimensiones idénticas. A los barcos y grúas no les importa qué hay dentro del contenedor; el método de enganche y apilamiento es universal."*

En el desarrollo de software sufríamos el mismo caos. Una app web requiere versiones específicas de librerías, bases de datos y configuraciones. Cuando pasamos el código a otra laptop o servidor, el sistema falla: el dilema de 'en mi máquina funciona'. Solomon Hykes fundó dotCloud y en 2013 liberó Docker. Docker empaqueta la aplicación con todo su runtime y dependencias en una caja estándar llamada contenedor, garantizando que corra igual en cualquier máquina."

Conceptos clave de la PPT a mencionar:

- Contenerización = empaquetar app + dependencias + configuración mínima
- Reduce diferencias entre desarrollo, pruebas y producción
- No elimina TODAS las diferencias (kernel, arquitectura, permisos aún importan)

Bloque 2: Virtualización e Hipervisores (Tipo 1 vs Tipo 2)

Guión Sugerido (Lo que debes decir): "Una máquina virtual ejecuta un sistema operativo completo sobre hardware virtualizado. Consume gigabytes de disco y RAM. Un contenedor no es una máquina virtual: comparte el kernel del sistema operativo del host. Es un proceso aislado, consumiendo solo megabytes y arrancando en milisegundos.

Los hipervisores se dividen en dos tipos: - **Tipo 1 (Bare Metal):** Corre directamente sobre el hardware (ej. Proxmox, VMware ESXi, Hyper-V Server). - **Tipo 2 (Hosted):** Corre como una aplicación sobre el SO host (ej. VirtualBox, VMware Workstation).

Docker NO es un hipervisor. Usa aislamiento a nivel de sistema operativo usando namespaces y cgroups."

Tabla de la PPT para proyectar:

Criterio	Máquina Virtual	Contenedor
Sistema operativo	Incluye SO completo	Comparte kernel del host
Arranque	Más lento	Rápido (milisegundos)
Consumo	Mayor RAM y disco	Menor consumo
Uso común	Laboratorios o servidores completos	Apps reproducibles
Aislamiento	Fuerte (kernel propio)	Proceso aislado (kernel compartido)

Bloque 3: Instalación de Docker (Desktop vs Engine)

Guión Sugerido (Lo que debes decir): "No memoricen comandos de instalación, ya que varían por sistema operativo. Sigan siempre la documentación oficial de Docker.

- Si usan **Windows o macOS**, instalen **Docker Desktop** (ya incluye el motor de Docker, la consola y Docker Compose integrado). En Windows requiere el componente **WSL2** (Windows Subsystem for Linux 2) para proveer un kernel de Linux ligero. - Si usan **Linux** o una **VM Linux**, instalen **Docker Engine** según la distribución y configuren los permisos post-instalación.

Para este curso, si necesitan un laboratorio limpio, se recomienda instalar una VM con Ubuntu Server: 2 CPU, 4 GB de RAM, 25 GB de disco y red tipo NAT."

Bloque 4: Imágenes y Contenedores (Diferencias clave)

Guión Sugerido (Lo que debes decir): "La diferencia entre imagen y contenedor es FUNDAMENTAL.

La **Imagen** es una plantilla inmutable de solo lectura con el sistema base, dependencias y código. El **Contenedor** es la instancia en ejecución creada a partir de esa imagen.

Regla mental: La imagen es la receta de cocina; el contenedor es el plato de comida ya servido y listo para comer.

El ciclo de vida básico: Obtener (docker pull) → Ejecutar (docker run) → Ver (docker ps) → Detener (docker stop)."

Bloque 5: Comandos Esenciales (Práctica en Consola)

Guión Sugerido (Lo que debes decir): "Para interactuar con Docker, abrimos la terminal. Si ejecutamos `docker run hello-world`, Docker verifica si tiene la imagen localmente. Si no, la descarga de Docker Hub (pull), crea el contenedor, ejecuta el saludo y se detiene.

Levantaremos un servidor web Nginx en el puerto 8080: `docker run --name web-demo -d -p 8080:80 nginx`. El flag `-d` corre en segundo plano (detached), `-p 8080:80` mapea el puerto 8080 de nuestra máquina al 80 interno del contenedor y `--name web-demo` le da un alias claro.

Podemos ver el contenedor activo con `docker ps` y todos con `docker ps -a`. Detenemos con `docker stop web-demo`, eliminamos con `docker rm web-demo` y revisamos las imágenes con `docker images`."

Bloque 6: Primera Aplicación Flask

Guión Sugerido (Lo que debes decir): "Ahora crearemos nuestra propia aplicación web. Escribiremos una app Flask sencilla con tres archivos:

1. Un `requirements.txt` con la dependencia: `flask==3.0.3`. 2. Un `app.py` que importará Flask, definirá la ruta base `/` que devolverá 'Hola Docker desde PIT 2026' y escuchará en el puerto 5000. 3. Un `Dockerfile`, la receta para fabricar la imagen:

- `FROM python:3.12-slim` — imagen base ligera de Python
- `WORKDIR /app` — directorio de trabajo interno
- `COPY requirements.txt .` — copia dependencias
- `RUN pip install --no-cache-dir -r requirements.txt` — instala librerías
- `COPY app.py .` — copia código fuente
- `EXPOSE 5000` — documenta puerto expuesto
- `CMD ["python", "app.py"]` — comando de inicio

Compilaremos la imagen con: `docker build -t mi-flask:v1 .` La ejecutaremos con: `docker run --name flask-demo -d -p 5000:5000 mi-flask:v1` Probamos en `http://localhost:5000` y finalmente limpiamos."

💬 Dinámica e Interacción en el Aula

- **Pregunta para romper el hielo:** "¿Qué pasa si actualizas la versión de Java o Node.js directamente en tu servidor de producción?"
 - *Respuesta:* Puede romper otras aplicaciones que dependan de versiones anteriores. Docker soluciona esto porque cada contenedor tiene su propio runtime aislado.
- **Pregunta técnica:** "¿Un contenedor de Linux puede correr sobre Windows de manera nativa?"
 - *Respuesta:* No de forma nativa. Docker Desktop en Windows requiere **WSL2** para proveer el Kernel de Linux real.

? Preguntas Frecuentes de los Alumnos (FAQs)

1. ¿Docker consume recursos si no hay contenedores activos?

- *Respuesta:* A diferencia de las VMs que consumen RAM fija desde que se encienden, Docker no consume CPU si los contenedores están apagados.

2. ¿Qué es la red NAT en una máquina virtual?

- *Respuesta:* Es una red que permite que la VM se conecte a internet usando la IP de la máquina física (Host), ideal para practicar sin conflictos de red.

3. ¿Qué pasa si hago docker run sin -d?

- *Respuesta:* El contenedor correrá en primer plano (modo attached), y la terminal quedará bloqueada mostrando los logs. Para salir se usa Ctrl+C.

Práctica en Consola Paso a Paso (Guía Visual del Docente)

Lab 1: Validación de Docker

```
docker --version
docker info
docker run hello-world
```

(Señalar las líneas 'Unable to find image... locally' para explicar el proceso de pull automático)

Lab 2: Desplegar Nginx

```
docker run --name web-demo -d -p 8080:80 nginx
docker ps
```

Abrir <http://localhost:8080> en el navegador. (Explicar la diferencia entre usar **-d** vs correr de manera interactiva)

Lab 3: Detener y Eliminar Nginx

```
docker stop web-demo
docker ps -a
docker rm web-demo
docker images
```

Lab 4: Construir y levantar la App Flask

```
cd codigo/sesion1
docker build -t mi-flask:v1 .
docker images
docker run --name flask-demo -d -p 5000:5000 mi-flask:v1
```

Validar abriendo `http://localhost:5000`

Lab 5: Limpieza Final

```
docker stop flask-demo
docker rm flask-demo
docker rmi mi-flask:v1
```

Taller Práctico / Ejercicio del Alumno

Taller A (Core): Construcción y Ejecución de App Flask

Los estudiantes deben clonar la carpeta de la clase, ingresar a `codigo/sesion1`, revisar los archivos del proyecto, compilar la imagen `mi-flask:v1` y validar su correcta ejecución en el puerto 5000.

Taller B (Complementario): Manipulación en Caliente y `docker cp`

Consigna para la Clase: Desplegar un servidor Apache HTTP (`httpd`) en el puerto 8081, crear un archivo `index.html` local con la frase "Bienvenido a PIT-UNI 2026", copiarlo dentro de Apache usando `docker cp`, y limpiar.

Solución Paso a Paso:

```
# 1. Levantar Apache
docker run --name apache-test -d -p 8081:80 httpd

# 2. Crear index.html local
echo "<h1>Bienvenido a PIT-UNI 2026</h1>" > index.html

# 3. Copiar archivo al contenedor
docker cp index.html apache-test:/usr/local/apache2/htdocs/index.html

# 4. Validar en http://localhost:8081

# 5. Limpiar
docker stop apache-test && docker rm apache-test
```

Trabajo para el Hogar

Desplegar dos servidores web en paralelo:

- Un servidor Nginx expuesto en el puerto `8080`.
- Un servidor Apache (`httpd`) expuesto en el puerto `8081`.
- Modificar la página principal de cada uno usando `docker cp` con textos personalizados ("Servidor 1: Nginx - UNI" y "Servidor 2: Apache - UNI").
- **Entregable:** Captura de pantalla de los dos navegadores mostrando la conexión concurrente.

Solucionario del Trabajo

```
docker run --name web-nginx -d -p 8080:80 nginx
docker run --name web-apache -d -p 8081:80 httpd
echo "Servidor 1: Nginx - UNI" > nginx.html
docker cp nginx.html web-nginx:/usr/share/nginx/html/index.html
echo "Servidor 2: Apache - UNI" > apache.html
docker cp apache.html web-apache:/usr/local/apache2/htdocs/index.html
# Validar en http://localhost:8080 y http://localhost:8081
```

⚠ Gestión del Aula y Errores Comunes

Error	Solución
<code>bind: address already in use</code>	El puerto ya está en uso. Detener la app local o cambiar el mapeo (ej. <code>-p 8082:80</code>)
No abre en navegador	Correr <code>docker ps -a</code> para ver si el contenedor se cayó y revisar <code>docker logs <nombre></code>
<code>image not found</code>	Verificar nombre e imagen con <code>docker images</code> . Corregir typos
Build fails	Asegurarse de estar en la carpeta correcta con <code>cd</code> y que el Dockerfile no tenga errores
<code>docker daemon is not running</code>	Iniciar Docker Desktop (Windows/macOS) o <code>sudo systemctl start docker</code> (Linux)

📄 Evaluación — Test de la Sesión 1 (12 Preguntas)

1. ¿Cuál es la diferencia fundamental de arquitectura entre una Máquina Virtual (VM) y un Contenedor de Docker?

- a) Las VMs son más ligeras y veloces al arrancar que los contenedores.
- b) Las VMs requieren empaquetar un Sistema Operativo Invitado completo encima de un hipervisor, mientras que los contenedores comparten el Kernel del Host.
- c) Los contenedores virtualizan el hardware físico del servidor y las VMs aíslan procesos.
- d) No existe diferencia de arquitectura, solo cambian las herramientas de consola.

2. Si ejecutas el comando `docker run -d -p 8080:80 nginx`, ¿qué acción realiza el flag `-p 8080:80`?

- a) Abre el puerto 80 del host y lo bloquea internamente.
- b) Enruta el tráfico del puerto 80 del Host hacia el puerto 8080 del contenedor.
- c) Enruta el tráfico del puerto 8080 del Host hacia el puerto 80 interno del contenedor.
- d) Establece los límites de descarga de red del contenedor.

3. **¿Qué comando utilizarías para verificar el ID, nombre y estado de los contenedores que se están ejecutando en este momento?**

- a) `docker container status`
- b) `docker image ls`
- c) `docker container ls` (o `docker ps`)
- d) `docker inspect --all`

4. **Si un contenedor con ID `abc123xyz` debe ser detenido y luego eliminado, ¿qué comandos se deben ejecutar?**

- a) `docker image rm abc123xyz`
- b) `docker container stop abc123xyz` y luego `docker container rm abc123xyz`
- c) `docker kill abc123xyz` y luego `docker container prune`
- d) `docker system prune abc123xyz`

5. **¿Qué sucede al ejecutar `docker run hello-world` si la imagen no está descargada localmente?**

- a) El comando falla inmediatamente y muestra un error de red.
- b) Docker busca la imagen localmente, y al no encontrarla, la descarga automáticamente de Docker Hub y la ejecuta.
- c) El comando crea una imagen vacía desde cero en tu computadora.
- d) El comando solicita permisos de root para compilar el código fuente.

6. **¿Cuál es la función de los Control Groups (cgroups) utilizados por Docker en Linux?**

- a) Asignar una dirección IP estática a cada contenedor.
- b) Limitar y controlar el consumo de recursos físicos (RAM, CPU, I/O de disco) que un contenedor puede utilizar.
- c) Dividir la visibilidad de los procesos internos del sistema.
- d) Cifrar los datos almacenados en los volúmenes locales.

7. **¿Qué namespace del Kernel de Linux utiliza Docker para garantizar que un contenedor tenga su propio stack de red aislado?**

- a) Namespace PID
- b) Namespace MNT
- c) Namespace NET
- d) Namespace IPC

8. **Si deseas eliminar una imagen llamada `nginx:alpine` para liberar espacio, ¿cuál es el comando correcto?**

- a) `docker container rm nginx:alpine`
- b) `docker image rm nginx:alpine` (o `docker rmi`)
- c) `docker system clean nginx:alpine`
- d) `docker network rm nginx:alpine`

9. **¿Qué acción realiza el flag `-d` (detached) cuando se ejecuta un contenedor?**

- a) Conecta el contenedor a internet por cable.

- b) Corre el contenedor en segundo plano, liberando la consola de inmediato.
- c) Abre una terminal interactiva tipo Bash.
- d) Desactiva los logs del contenedor para ahorrar memoria.

10. ¿Bajo qué arquitectura opera el motor de Docker para comunicarse entre la CLI y el daemon?

- a) Arquitectura monolítica local.
- b) Arquitectura Cliente-Servidor comunicada mediante sockets UNIX o redes locales con API REST.
- c) Comunicación por archivos de texto compartidos.
- d) Remotas SSH sin cifrado de datos.

11. ¿Qué instrucción del Dockerfile define el comando que se ejecutará cuando arranca el contenedor?

- a) RUN
- b) COPY
- c) CMD
- d) FROM

12. ¿Por qué Docker Desktop en Windows necesita obligatoriamente el componente WSL2?

- a) Porque WSL2 proporciona una interfaz gráfica mejorada para Docker.
- b) Porque WSL2 provee un Kernel de Linux real necesario para ejecutar los Namespaces y cgroups que requiere Docker.
- c) Porque sin WSL2 no se puede instalar Python en Windows.
- d) Porque WSL2 actúa como un firewall para proteger los contenedores.

 Solucionario Sesión 1

1-b | 2-c | 3-c | 4-b | 5-b | 6-b | 7-c | 8-b | 9-b | 10-b | 11-c | 12-b